CliniqAI Phase 1 â€” Deep Line-by-Line Explanation

This document explains **every single line** of Phase 1 code using 3 layers:

1. **Story** â€” Real-world analogy (no tech jargon)

2. **Technical** â€” What the code actually does

3. **Example** â€” Concrete example with real data

PART 1: DRUG CONFLICT CHECKER (alert_tool.py)

What is an "Allergy Family"?

LAYER 1 â€” The Story:

Imagine you're allergic to peanuts. Peanuts are in the "legume" family. Other legumes include lentils, chickpeas, soybeans. If you're allergic to peanuts, you might also be allergic to other legumes. So doctors group them together.

In CliniqAI, we do the same with medicines. Penicillin is one drug, but there are 8 other drugs that are chemically similar (amoxicillin, ampicillin, etc.). They're all in the "penicillin family." If a patient is allergic to penicillin, they're probably allergic to all of them.

LAYER 2 â€” Technical:

```
ALLERGY_FAMILIES = { "penicillin": [ "amoxicillin", "ampicillin", "augmentin", "amoxyclav", "cloxacillin", "flucloxacillin", "piperacillin", "co-amoxiclav" ], "cephalosporin": [ "cefalexin", "cefuroxime", "cefixime", "ceftriaxone", "cefpodoxime", "cefdinir", "cefadroxil" ], ... }
```

This is a **dictionary** (key-value pairs):

- **Key** = family name (e.g., "penicillin")

- **Value** = list of drugs in that family

LAYER 3 â€” Example:

```
Patient's allergy: "penicillin" New prescription: "Amoxicillin" We check: Is "amoxicillin" in ALLERGY_FAMILIES["penicillin"]? Answer: YES â†’ ALERT!
```

Understanding the 3-Layer Check (Line by Line)

LAYER 1: Direct Allergy Check

The Story:

A pharmacist checks: "Is the patient allergic to this exact drug family?"

The Code:

```
# â”€â”€ Layer 1: Direct allergy check â”€â”€ for allergy in patient_allergies: # Line 1: Loop through each allergy allergy_lower = allergy.lower() # Line 2: Convert to lowercase for family, drugs in ALLERGY_FAMILIES.items(): # Line 3: Loop through each drug family # Check if the allergy matches a family name or a specific drug in the family if allergy_lower == family or allergy_lower in drugs: #
```

```
Line 4: Does allergy match?
```

```
for new_name in new_med_names: # Line 5: Loop through new medicines if any(drug in new_name for drug in drugs): # Line 6: Is new med in this family? alerts.append({ # Line 7: Add alert "severity": "HIGH", "type": "ALLERGY", "message": ( f"ALLERGY ALERT: Patient is allergic to {allergy}. " f"New prescri
```

```
ption includes '{new_name}' " f"which belongs to the {family} family." ) })
```

Line-by-Line Breakdown:

Line 1: `for allergy in patient_allergies:`

- **What:** Loop through each allergy the patient has

- **Input:** `patient_allergies` = list like `["penicillin", "sulfa"]`

- **Example:** First iteration: `allergy = "penicillin"`

Line 2: `allergy_lower = allergy.lower()`

- **What:** Convert the allergy name to lowercase

- **Why:** User might type "PENICILLIN" or "Penicillin" â€” we want to match "penicillin"

- **Example:** `"PENICILLIN".lower()` â†’ `"penicillin"`

Line 3: `for family, drugs in ALLERGY_FAMILIES.items():`

- **What:** Loop through the dictionary. Each iteration gives us:

- `family` = the key (e.g., "penicillin")

- `drugs` = the value (e.g., ["amoxicillin", "ampicillin", ...])

- **Example:**

```

First iteration:

family = "penicillin"

drugs = ["amoxicillin", "ampicillin", "augmentin", ...]

Second iteration:

family = "cephalosporin"

drugs = ["cefalexin", "cefuroxime", ...]

```

Line 4: `if allergy_lower == family or allergy_lower in drugs:`

- **What:** Check if the patient's allergy matches this drug family in TWO ways:

1. **Exact match:** `allergy_lower == family` â€” patient allergic to "penicillin" and family is "penicillin"

2. **Specific drug match:** `allergy_lower in drugs` â€” patient allergic to "amoxicillin" (a specific drug) and we're checking the penicillin family

- **Example:**

```

If allergy = "penicillin" and family = "penicillin":
```

"penicillin" == "penicillin" â†' TRUE â†' Continue

If allergy = "amoxicillin" and family = "penicillin":

"amoxicillin" == "penicillin" â†' FALSE

"amoxicillin" in ["amoxicillin", "ampicillin", ...] â†' TRUE â†' Continue

```

**Line 5: `for new_name in new_med_names:`**

- **What:** Loop through the medicines in the NEW prescription
- **Input:** `new_med_names` = list like `["amoxicillin", "paracetamol"]`
- **Example:** First iteration: `new_name = "amoxicillin"`

**Line 6: `if any(drug in new_name for drug in drugs):`**

- **What:** Check if ANY drug from this family appears in the new medicine name
- **Breaking it down:**
- `for drug in drugs` â€" loop through each drug in the family (e.g., "amoxicillin", "ampicillin")
- `drug in new_name` â€" does this drug name appear as a substring in the new medicine?
- `any(...)` â€" if ANY of them match, return True
- **Example:**

```

drugs = ["amoxicillin", "ampicillin", "augmentin", ...]

new_name = "amoxicillin"

Check:

"amoxicillin" in "amoxicillin" â†' TRUE

Result: any([True, False, False, ...]) â†' TRUE

```

**Line 7: `alerts.append({...})`**

- **What:** Add a new alert to the list
- **The alert contains:**
- `"severity": "HIGH"` â€" This is dangerous
- `"type": "ALLERGY"` â€" It's an allergy conflict
- `"message"` â€" Human-readable explanation

---

### LAYER 2: Cross-Allergy Check

**The Story:**

A pharmacist knows: "If you're allergic to penicillin, there's a 10% chance you're also allergic to cephalosporins (a related family)." This is called cross-reactivity. We warn about it, but it's not as dangerous as a direct allergy.
```

The Code:

```
# â”€â”€ Layer 2: Cross-allergy check â”€â”€ for allergy in patient_allergies: # Line 1: Loop through allergies allergy_lower = allergy.lower() # Line 2: Lowercase if allergy_lower in CROSS_ALLERGY_WARNINGS: # Line 3: Is this allergy in the warnings table? rule = CROSS_ALLERGY_WARNINGS[allergy_lower] # Line 4: Get the warning rule for related_family in rule["families"]: # Line 5: Loop through related families relat
```

```
ed_drugs = ALLERGY_FAMILIES.get(related_family, []) # Line 6: Get drugs in related family for new_name in new_med_names: # Line 7: Loop through new medicines if any(drug in new_name for drug in related_drugs): # Line 8: Match? alerts.append({ # Line 9: Add MEDIUM alert "severity": "MEDIUM", "type": "CROSS_ALLERGY", "message": (
```

```
f"CROSS-ALLERGY WARNING: Patient is allergic to {allergy}. " f"'{new_name}' is in a related family ({related_family}). " f"{rule['message']}" ) })
```

What is CROSS_ALLERGY_WARNINGS?

```
CROSS_ALLERGY_WARNINGS = { "penicillin": { "families": ["cephalosporin"], "message": "Possible cross-reactivity (~10%). Use with caution." }, "nsaid": { "families": ["aspirin"], "message": "Aspirin belongs to the same anti-inflammatory group." } }
```

Line-by-Line:

Line 3: `if allergy_lower in CROSS_ALLERGY_WARNINGS:`

- **What:** Check if this allergy has known cross-reactivity with other families

- **Example:**

```

If allergy = "penicillin":

"penicillin" in CROSS_ALLERGY_WARNINGS â†’ TRUE

If allergy = "sulfa":

"sulfa" in CROSS_ALLERGY_WARNINGS â†’ FALSE (not in the table)

```

Line 4: `rule = CROSS_ALLERGY_WARNINGS[allergy_lower]`

- **What:** Get the cross-allergy rule for this allergy

- **Example:**

```

rule = {

"families": ["cephalosporin"],

"message": "Possible cross-reactivity (~10%). Use with caution."

}

```

Line 5: `for related_family in rule["families"]:`

- **What:** Loop through related families (e.g., ["cephalosporin"])

- **Example:** `related_family = "cephalosporin"`

Line 6: `related_drugs = ALLERGY_FAMILIES.get(related_family, [])`

- **What:** Get the list of drugs in the related family

- **The `.get()` method:** If the family exists, return its drugs. If not, return empty list `[]`

- **Example:**

```

related_drugs = ALLERGY_FAMILIES.get("cephalosporin", [])

# Returns: ["cefalexin", "cefuroxime", "cefixime", ...]

```

Lines 7-9: Same as Layer 1, but with `"severity": "MEDIUM"` instead of `"HIGH"`

LAYER 3: Drug-Drug Interaction Check

The Story:

A pharmacist checks: "Are there any two medicines in this prescription that are known to be dangerous together?" For example, Warfarin (blood thinner) + Aspirin (also thins blood) = too much thinning = bleeding risk.

The Code:

```
# â”€â”€ Layer 3: Drug-drug interaction check â”€â”€ all_meds = current_med_names + new_med_names # Line 1: Combine old + new medicines for (drug_a, drug_b), (severity, message) in DANGEROUS_COMBOS.items(): # Line 2: Loop through known bad pairs a_present = any(drug_a in med for med in all_meds) # Line 3: Is drug_a in any medicine? b_present = any(drug_b in med for med in all_meds) # Line 4: Is drug_b in any medicine? if a_present and b_present:
```

```
# Line 5: Are BOTH present? alerts.append({ # Line 6: Add alert "severity": severity, "type": "INTERACTION", "message": f"DRUG INTERACTION ({severity}): {message}" })
```

What is DANGEROUS_COMBOS?

```
DANGEROUS_COMBOS = { ("warfarin", "aspirin"): ("HIGH", "Serious bleeding risk â€” combined anticoagulation"), ("warfarin", "nsaid"): ("HIGH", "NSAIDs increase bleeding risk with warfarin"), ("metformin", "contrast"): ("HIGH", "Hold metformin before contrast procedures â€” lactic acidosis risk"), ... }
```

Line-by-Line:

Line 1: `all_meds = current_med_names + new_med_names`

- **What:** Combine the medicines the patient is already taking with the new prescription

- **Example:**

```

current_med_names = ["warfarin", "metformin"]

new_med_names = ["aspirin", "paracetamol"]

all_meds = ["warfarin", "metformin", "aspirin", "paracetamol"]

```

Line 2: `for (drug_a, drug_b), (severity, message) in DANGEROUS_COMBOS.items():`

- **What:** Loop through the dictionary of dangerous pairs

- **Unpacking:** Each item has:

- Key = tuple of two drugs: `(drug_a, drug_b)` e.g., `("warfarin", "aspirin")`

- Value = tuple of severity and message: `(severity, message)` e.g., `("HIGH", "Serious bleeding risk...")`

- **Example:**

```

First iteration:

drug_a = "warfarin"

drug_b = "aspirin"

severity = "HIGH"

message = "Serious bleeding risk â€" combined anticoagulation"

```

Line 3: `a_present = any(drug_a in med for med in all_meds)`

- **What:** Check if drug_a appears in ANY of the medicines

- **Breaking it down:**

- `for med in all_meds` â€" loop through each medicine name

- `drug_a in med` â€" does drug_a appear as a substring?

- `any(...)` â€" if ANY match, return True

- **Example:**

```

drug_a = "warfarin"

all_meds = ["warfarin", "metformin", "aspirin", "paracetamol"]

Check:

"warfarin" in "warfarin" â†' TRUE

Result: any([TRUE, FALSE, FALSE, FALSE]) â†' TRUE

```

Line 4: `b_present = any(drug_b in med for med in all_meds)`

- Same as Line 3, but for drug_b

- **Example:**

```

drug_b = "aspirin"

"aspirin" in "aspirin" â†' TRUE

```

Line 5: `if a_present and b_present:`

- **What:** Only alert if BOTH drugs are present

- **Example:**

```

If a_present = TRUE and b_present = TRUE â†' Alert!

If a_present = TRUE and b_present = FALSE â†' No alert (safe)

```

Line 6: Add the alert with the severity and message from the dictionary

---

Final Step: Sort and Return

```
# Sort: HIGH severity first alerts.sort(key=lambda x: 0 if x["severity"] == "HIGH" else 1) return {
"has_alerts": len(alerts) > 0, "alert_count": len(alerts), "high_severity": sum(1 for a in alerts if
a["severity"] == "HIGH"), "alerts": alerts }
```

Line 1: `alerts.sort(key=lambda x: 0 if x["severity"] == "HIGH" else 1)`

- **What:** Sort alerts so HIGH severity appears first

- **How:**

- If severity is "HIGH", assign key = 0

- Otherwise, assign key = 1

- Python sorts by key in ascending order, so 0 comes before 1

- **Example:**

```

Before: [MEDIUM alert, HIGH alert, MEDIUM alert]

After: [HIGH alert, MEDIUM alert, MEDIUM alert]

```

Return dictionary:

- `has_alerts` â€" True if any alerts, False otherwise

- `alert_count` â€" Total number of alerts

- `high_severity` â€" Count of HIGH severity alerts

- `alerts` â€" The actual alert list

PART 2: VISION TOOL (vision_tool.py)

Understanding Image Processing

What is `Image.open(io.BytesIO(image_bytes))`?

LAYER 1 â€” The Story:

Imagine you receive a photo as a stream of binary data (1s and 0s). You need to convert it into something you can actually look at â€” an image object. That's what this line does.

LAYER 2 â€” Technical:

```
try: image = Image.open(io.BytesIO(image_bytes)) except Exception as e: return {"error": f"Could not read image file: {str(e)}. Please upload a valid JPG or PNG."}
```

Breaking it down:

`io.BytesIO(image_bytes)`

- **What:** Convert raw bytes into a file-like object

- **Why:** PIL's `Image.open()` expects a file path or file-like object, not raw bytes

- **Example:**

```

image_bytes = b'\x89PNG\r\n\x1a\n...' (raw binary data from upload)

io.BytesIO(image_bytes) â†' Creates a virtual file in memory
```

`Image.open(...)`

- **What:** Open the image and validate it

- **Returns:** An Image object that PIL can work with

- **Throws error if:** The bytes are not a valid image (e.g., a .txt file)

`try...except`

- **What:** Try to open the image. If it fails, catch the error and return a helpful message

- **Why:** If someone uploads a .txt file, `Image.open()` throws `UnidentifiedImageError`. We catch it and tell the user what went wrong

- **Example:**

```

User uploads: document.txt

Image.open() throws: UnidentifiedImageError

We catch it and return:

{"error": "Could not read image file: cannot identify image file. Please upload a valid JPG or PNG."}
```

Understanding Text Processing

What is `text.startswith()` and `strip()`?

The Code:

```
text = response.text.strip() if text.startswith("```json"): text = text[7:-3].strip() elif
text.startswith("```"): text = text[3:-3].strip()
```

LAYER 1 â€” The Story:

Gemini sometimes wraps JSON in markdown code blocks (like this):

```
{"patient_name": "Ramesh", ...}
```

We need to remove those wrapper lines to get just the JSON.

LAYER 2 â€” Technical:

`text.strip()`

- **What:** Remove whitespace from the beginning and end of the string

- **Why:** Gemini might add extra spaces or newlines

- **Example:**

```

Before: " \n ```json\n {...} \n ```\n "

After: "```json\n {...}\n ```"

```

`text.startswith("```json")`

- **What:** Check if the text starts with the string `"```json"`

- **Returns:** True or False

- **Example:**

```

If text = "```json\n{...}\n```"

text.startswith("```json") â†' TRUE

If text = "```\n{...}\n```"

text.startswith("```json") â†' FALSE

```

`text[7:-3]`

- **What:** Extract a substring from position 7 to the end minus 3 characters

- **Why:** Remove the markdown wrapper

- **Example:**

```

Original: "```json\n{...}\n```"

Positions: 0123456789...

text[7:-3] removes:
```

- First 7 chars: "```json" (7 characters)

- Last 3 chars: "```" (3 characters)

Result: "\n{...}\n"

```

**`text[3:-3]`**

- **What:** Same as above, but for ` "```" ` (3 characters, not 7)

- **Example:**

```

Original: "```\n{...}\n```"

text[3:-3] â†’ "\n{...}\n"

```

**Final `strip()`**

- **What:** Remove any remaining whitespace

- **Example:**

```

Before: "\n{...}\n"

After: "{...}"

```

---

### What is `json.loads()`?

**The Code:**

```
try: data = json.loads(text) return data except Exception as e: return {"error": f"Failed to parse JSON: {str(e)}", "raw_response": response.text}
```

**LAYER 1 â€” The Story:**

You have a string that looks like JSON: `'{"name": "Ramesh", "age": 45}'`. You want to convert it into a Python dictionary so you can access fields like `data["name"]`. That's what `json.loads()` does.

**LAYER 2 â€” Technical:**

**`json.loads(text)`**

- **What:** Parse a JSON string and convert it to a Python dictionary

- **Input:** A string like `'{"name": "Ramesh", "age": 45}'`

- **Output:** A dictionary like `{"name": "Ramesh", "age": 45}`

- **Throws error if:** The string is not valid JSON (e.g., missing quotes, trailing commas)

**`try...except`**

- **What:** Try to parse. If it fails, catch the error
```

- **Why:** Gemini sometimes returns invalid JSON (e.g., with comments or trailing commas)

- **Example:**

```

Gemini returns: '{"name": "Ramesh", "age": 45,}' (trailing comma â€” invalid!)

json.loads() throws: JSONDecodeError

We catch it and return:

{"error": "Failed to parse JSON: Expecting value...", "raw_response": "..."}

```

Complete Example:

```
# Gemini returns this text: response_text = '''```json { "patient_name": "Ramesh Gupta",
"patient_age": 45, "medicines": [{"name": "Metformin", "dose": "500mg"}] }
```

Step 1: Strip whitespace

text = response_text.strip()

Result: "```json\n{...}\n```"

Step 2: Check if it starts with "```json"

if text.startswith("```json"):

Yes, so remove first 7 and last 3 chars

text = text[7:-3].strip()

Result: "{...}"

Step 3: Parse JSON

data = json.loads(text)

Result: {"patient_name": "Ramesh Gupta", "patient_age": 45, ...}

Step 4: Return the dictionary

return data

```
--- --- ## PART 3: SERVER.PY (Line by Line) ### Understanding the Complete Flow Let me explain
`server.py` from top to bottom, showing where each variable comes from. #### Imports and Setup
```

import os

import re

import uuid

from datetime import datetime, date

from fastapi import FastAPI, UploadFile, File

from fastapi.middleware.cors import CORSMiddleware

from pydantic import BaseModel

from pymongo import MongoClient

from dotenv import load_dotenv

from agent.tools.vision_tool import extract_from_prescription

from agent.tools.alert_tool import check_drug_conflicts

```
**What each import does:** | Import | What it is | Why we use it | |---|---|---| | `os` | Operating system module | Read environment variables from `.env` | | `re` | Regular expressions | Escape special characters in user queries | | `uuid` | Generate unique IDs | Create unique patient IDs | | `datetime, date` | Date/time handling | Record visit dates | | `FastAPI` | Web framework | Create the server | | `UploadFile, File` | File upload handling | Accept image uploads from users | | `CORSMiddl
```

```
eware` | Cross-Origin Resource Sharing | Allow frontend to call backend | | `BaseModel` | Data validation | Validate incoming JSON requests | | `MongoClient` | MongoDB connection | Connect to the database | | `load_dotenv` | Load .env file | Read API keys from `.env` | | `extract_from_prescription` | Our vision tool | Read prescription photos | | `check_drug_conflicts` | Our alert tool | Check for drug conflicts | --- #### Loading Environment Variables
```

load_dotenv()

MONGODB_URI = os.getenv("MONGODB_URI")

GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")

```
**What happens:** 1. **`load_dotenv()`** â€” Reads the `.env` file and loads all variables into `os.environ` 2. **`os.getenv("MONGODB_URI")`** â€” Gets the value of the `MONGODB_URI` variable from `.env` 3. **`os.getenv("GOOGLE_API_KEY")`** â€” Gets the value of the `GOOGLE_API_KEY` variable from `.env` **Example `.env` file:**
```

GOOGLE_API_KEY=sk-proj-abc123...

MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/cliniqai

```
**After `load_dotenv()`:**
```

MONGODB_URI = "mongodb+srv://user:pass@cluster.mongodb.net/cliniqai"

GOOGLE_API_KEY = "sk-proj-abc123..."

```
--- #### MongoDB Connection (Lazy Loading)
```

client = None

db = None

patients_collection = None

def get_db():

"""Connect to MongoDB on first use. Returns True if connected."""

global client, db, patients_collection

if patients_collection is not None:

return True

if not MONGODB_URI or "youruser" in MONGODB_URI:

return False

try:

client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=5000)

client.admin.command("ping")

db = client["cliniqai"]

patients_collection = db["patients"]

return True

except Exception as e:

print(f"MongoDB connection failed: {e}")

return False

```
**What this does:** **`client = None, db = None, patients_collection = None`** - Start with empty variables - Don't connect yet (lazy loading) **`def get_db():`** - A function that connects to MongoDB when first called - Returns `True` if connected, `False` if not **`global client, db, patients_collection`** - Tell Python: "These variables are global (not local to this function)" - So we can modify them from inside the function **`if patients_collection is not None: return True`** - If alre
```

```
ady connected, don't connect again - Just return True **`if not MONGODB_URI or "youruser" in MONGODB_URI: return False`** - If MongoDB URI is not set or is a placeholder, return False - This allows the app to run without MongoDB (using in-memory storage) **`client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=5000)`** - Connect to MongoDB - `serverSelectionTimeoutMS=5000` means wait max 5 seconds **`client.admin.command("ping")`** - Test the connection by sending a "ping" command - If t
```

```
his fails, an exception is thrown and caught **`db = client["cliniqai"]`** - Get the database named "cliniqai" **`patients_collection = db["patients"]`** - Get the collection (table) named "patients" from the database --- #### FastAPI App Setup
```

app = FastAPI(title="CliniqAI", version="1.0")

app.add_middleware(

CORSMiddleware,

allow_origins=["*"],

allow_methods=["*"],

allow_headers=["*"],

)

```
**What this does:** **`app = FastAPI(...)`** - Create a FastAPI application - `title="CliniqAI"` â€” Name shown in API docs - `version="1.0"` â€” Version number **`add_middleware(CORSMiddleware, ...)`** - Allow requests from any origin (frontend) - Without this, the browser blocks requests from `localhost:3000` to `localhost:8000` --- #### Request Models
```

class QueryRequest(BaseModel):

query: str

```
**What this does:** - Define the shape of incoming JSON requests - When someone POSTs to `/query`, they must send: `{"query": "some text"}` - FastAPI automatically validates and converts it to a Python object **Example:**
```

User sends:

POST /query

{"query": "how many patients"}

FastAPI converts to:

request = QueryRequest(query="how many patients")

We access it as:

request.query # "how many patients"

```
--- #### Health Check Endpoint
```

@app.get("/health")

def health():

"""Simple health check endpoint."""

db_connected = get_db()

if db_connected:

mongo_status = "connected"

else:

mongo_status = "not configured (using in-memory store)"

return {

"status": "running",

"mongodb": mongo_status,

"google_api_key_set": bool(GOOGLE_API_KEY and "your_google" not in GOOGLE_API_KEY),

}

```
**What this does:** **`@app.get("/health")`** - Create a GET endpoint at `/health` - When user visits
`http://localhost:8000/health`, this function runs **`db_connected = get_db()`** - Try to connect to
MongoDB - Returns True or False **`if db_connected: mongo_status = "connected" else: mongo_status =
"not configured..."`** - Set the status message **`return {...}`** - Return a JSON response with: -
`status` â€” Is the server running? - `mongodb` â€” Is MongoDB connected? - `google_ap
```

```
i_key_set` â€” Is the API key configured? **Example response:**
```

{

"status": "running",

"mongodb": "not configured (using in-memory store)",

"google_api_key_set": false

}

```
--- #### Process Document Endpoint (The Main One)
```

@app.post("/process")

async def process_document(file: UploadFile = File(...)):

"""

1. Read uploaded image

2. Extract patient data using Gemini Vision

3. Check if patient already exists in MongoDB

4. Insert or update patient record

5. Run drug conflict check

6. Return everything to the frontend

"""

Step 1: Read image bytes

image_bytes = await file.read()

```
**What this does:** **`@app.post("/process")`** - Create a POST endpoint at `/process` - User uploads a file here **`async def process_document(file: UploadFile = File(...))`** - `async` â€” This function can run concurrently (handle multiple requests) - `file: UploadFile` â€” The uploaded file - `= File(...)` â€” FastAPI knows this is a file upload **`image_bytes = await file.read()`** - Read the file contents as bytes - `await` â€” Wait for the file to be fully read - `image_bytes` â€” Raw
```

```
binary data of the image **Example:**
```

User uploads: prescription.jpg

file.read() returns: b'\x89PNG\r\n\x1a\n...' (binary data)

```
--- #### Extract Data from Image
```

Step 2: Extract data from the prescription image

extracted = extract_from_prescription(image_bytes)

if "error" in extracted:

return {"error": extracted["error"], "raw": extracted.get("raw_response", "")}

```
**What this does:** **`extracted = extract_from_prescription(image_bytes)`** - Call the vision tool - Pass the image bytes - Get back a dictionary with patient data **Example return:**
```

{

"patient_name": "Ramesh Gupta",

"patient_age": 45,

"patient_gender": "Male",

"visit_date": "2026-05-16",

"doctor_name": "Dr. Sharma",

"medicines": [

{"name": "Metformin", "dose": "500mg", "frequency": "twice daily", "duration": "30 days"},

{"name": "Amlodipine", "dose": "5mg", "frequency": "once daily", "duration": "30 days"}

],

"allergies_mentioned": ["penicillin"],

"diagnosis": ["Type 2 Diabetes", "Hypertension"],

"tests_ordered": ["HbA1c", "Lipid Profile"],

"notes": "Follow up in 1 month"

}

```
**`if "error" in extracted:`** - Check if the vision tool returned an error - If yes, return the error immediately (don't continue) **Example error:**
```

{

"error": "GOOGLE_API_KEY not configured. Set it in .env file.",

"raw": ""

}

```
--- #### Check if Patient Exists
```

Step 3: Check if patient already exists (search by name)

patient_name = extracted.get("patient_name", "Unknown")

use_mongo = get_db()

Build the visit record from extracted data

visit = {

"date": extracted.get("visit_date", str(date.today())),

"doctor": extracted.get("doctor_name"),

"clinic": extracted.get("clinic_name"),

"diagnosis": extracted.get("diagnosis", []),

"medicines": extracted.get("medicines", []),

"tests": extracted.get("tests_ordered", []),

"notes": extracted.get("notes"),

}

```
**What this does:** **`patient_name = extracted.get("patient_name", "Unknown")`** - Get the patient name from the extracted data - If not present, use "Unknown" as default **`use_mongo = get_db()`** - Try to connect to MongoDB - Returns True or False **`visit = {...}`** - Create a visit record with all the extracted data - This will be added to the patient's document **Example:**
```

visit = {

"date": "2026-05-16",

"doctor": "Dr. Sharma",

"clinic": "City Clinic",

"diagnosis": ["Type 2 Diabetes"],

"medicines": [{"name": "Metformin", ...}],

"tests": ["HbA1c"],

"notes": "Follow up in 1 month"

}

```
--- #### MongoDB Path (If Connected)
```

if use_mongo:

â”€â”€ MongoDB path â”€â”€

existing_patient = patients_collection.find_one({"name": patient_name})

if existing_patient:

Step 4a: Patient exists â€” add new visit to their record

is_returning = True

existing_allergies = existing_patient.get("known_allergies", [])

new_allergies = extracted.get("allergies_mentioned", [])

all_allergies = list(set(existing_allergies + new_allergies))

patients_collection.update_one(

{"_id": existing_patient["_id"]},

{"$push": {"visits": visit}, "$set": {"known_allergies": all_allergies}}

)

patient_id = str(existing_patient["_id"])

visit_count = len(existing_patient.get("visits", [])) + 1

patient_allergies = all_allergies

for prev_visit in existing_patient.get("visits", []):

current_medicines.extend(prev_visit.get("medicines", []))

What this does: **`existing_patient = patients_collection.find_one({"name": patient_name})`** - Search MongoDB for a patient with this name - Returns the patient document if found, or None if not found **Example return:**

{

"_id": ObjectId("..."),

"patient_id": "uuid-123",

"name": "Ramesh Gupta",

"age": 45,

"known_allergies": ["penicillin"],

"conditions": ["Type 2 Diabetes"],

"visits": [

{"date": "2026-05-01", "doctor": "Dr. Sharma", ...},

{"date": "2026-05-10", "doctor": "Dr. Patel", ...}

]

}

`if existing_patient:` - If patient was found, this is a returning patient **`is_returning = True`** - Mark this as a returning patient **`existing_allergies = existing_patient.get("known_allergies", [])`** - Get allergies from the existing record - Default to empty list if not present **`new_allergies = extracted.get("allergies_mentioned", [])`** - Get allergies from the new prescription **`all_allergies = list(set(existing_allergies + new_allergies))`** - Combine both lists and remove

duplicates - `set()` removes duplicates, `list()` converts back to list - Example: ```python existing_allergies = ["penicillin"] new_allergies = ["penicillin", "sulfa"] all_allergies = list(set(["penicillin", "penicillin", "sulfa"])) # Result: ["penicillin", "sulfa"] ``` **`patients_collection.update_one(...)`** - Update the patient record in MongoDB - `{"_id": existing_patient["_id"]}` â€” Find the patient by ID - `{"$push": {"visits": visit}, "$set": {"known_allergies": all_al

```
lergies}}` â€” Two operations: - `$push` â€” Append the new visit to the visits array - `$set` â€” Update the allergies list **`patient_id = str(existing_patient["_id"])`** - Get the patient's ID (convert to string) **`visit_count = len(existing_patient.get("visits", [])) + 1`** - Count the number of visits - Add 1 because we're about to add a new visit **`patient_allergies = all_allergies`** - Use the combined allergies list for the alert check **`for prev_visit in existing_patient.get(
```

```
"visits", []): current_medicines.extend(...)`** - Get all medicines from previous visits - We need this to check drug-drug interactions --- #### In-Memory Path (If Not Connected)
```

else:

â”€â”€ In-memory fallback â”€â”€

existing_patient = next((p for p in in_memory_patients if p["name"] == patient_name), None)

if existing_patient:

is_returning = True

existing_allergies = existing_patient.get("known_allergies", [])

new_allergies = extracted.get("allergies_mentioned", [])

all_allergies = list(set(existing_allergies + new_allergies))

existing_patient["known_allergies"] = all_allergies

existing_patient["visits"].append(visit)

patient_id = existing_patient["patient_id"]

visit_count = len(existing_patient["visits"])

patient_allergies = all_allergies

for prev_visit in existing_patient["visits"][:-1]:

current_medicines.extend(prev_visit.get("medicines", []))

```
**What this does:** **`existing_patient = next((p for p in in_memory_patients if p["name"] == patient_name), None)`** - Search the in-memory list for a patient with this name - `next(...)` returns the first match or None if not found - This is equivalent to: ```python existing_patient = None for p in in_memory_patients: if p["name"] == patient_name: existing_patient = p break ``` **Rest is similar to MongoDB path, but:** - `existing_patient["visits"].append(v
```

```
isit)` â€” Append to the list (not MongoDB $push) - `existing_patient["visits"][:-1]` â€” All visits except the last one (which we just added) --- #### Run Drug Conflict Check
```

Step 5: Run drug conflict check

new_medicines = extracted.get("medicines", [])

conflict_result = check_drug_conflicts(

patient_allergies=patient_allergies,

current_medicines=current_medicines,

new_medicines=new_medicines,

)

```
**What this does:** **`new_medicines = extracted.get("medicines", [])`** - Get the medicines from the new prescription **`conflict_result = check_drug_conflicts(...)`** - Call the alert tool - Pass: - `patient_allergies` â€” All allergies (old + new) - `current_medicines` â€” Medicines from previous visits - `new_medicines` â€” Medicines in this prescription - Get back: ```python { "has_alerts": True, "alert_count": 1, "high_severity": 1, "alerts": [
```

```
{ "severity": "HIGH", "type": "ALLERGY", "message": "ALLERGY ALERT: Patient is allergic to penicillin. New prescription includes 'amoxicillin'..." } ] } ``` --- #### Return Response
```

Step 6: Return the result to the frontend

return {

"record_id": patient_id,

"is_returning": is_returning,

"patient": {

"name": patient_name,

"age": extracted.get("patient_age"),

"gender": extracted.get("patient_gender"),

"doctor": extracted.get("doctor_name"),

"visit_date": extracted.get("visit_date", str(date.today())),

"diagnosis": extracted.get("diagnosis", []),

"medicines": new_medicines,

"known_allergies": patient_allergies,

"visit_count": visit_count,

},

"alerts": conflict_result["alerts"],

}

```
**What this does:** Return a JSON response with everything the frontend needs: - `record_id` â€” The patient's ID in the database - `is_returning` â€” Is this a returning patient? - `patient` â€” All patient details - `alerts` â€” Any drug conflicts found **Example response:**
```

{

"record_id": "507f1f77bcf86cd799439011",

"is_returning": false,

"patient": {

"name": "Ramesh Gupta",

"age": 45,

"gender": "Male",

"doctor": "Dr. Sharma",

"visit_date": "2026-05-16",

"diagnosis": ["Type 2 Diabetes", "Hypertension"],

"medicines": [

{"name": "Metformin", "dose": "500mg", ...},

{"name": "Amlodipine", "dose": "5mg", ...}

],

"known_allergies": ["penicillin"],

"visit_count": 1

},

"alerts": []

}

```
--- ### Query Endpoint
```

@app.post("/query")

async def run_query(request: QueryRequest):

"""

Simple natural language query handler.

For Phase 1, we do basic keyword matching.

"""

query = request.query.lower()

use_mongo = get_db()

if use_mongo:

â”€â”€ MongoDB queries â”€â”€

if "how many" in query or "count" in query:

count = patients_collection.count_documents({})

return {"answer": f"Total patients in database: {count}"}

...

```
**What this does:** **`query = request.query.lower()`** - Get the user's query and convert to
lowercase - Example: "How many patients?" â†’ "how many patients?" **`if "how many" in query or
"count" in query:`** - Check if the query is asking for a count - If yes, count all documents in
MongoDB **`patients_collection.count_documents({})`** - Count all documents (empty filter `{}` means
"all") **Return:**
```

{"answer": "Total patients in database: 5"}

```
--- --- ## Summary: Where Do Variables Come From? | Variable | Where it comes from | Example |
|---|---|---| | `image_bytes` | User uploads a file â†’ `file.read()` | `b'\x89PNG...'` | |
`extracted` | Gemini Vision processes the image â†’ `extract_from_prescription()` |
`{"patient_name": "Ramesh", ...}` | | `patient_name` | Extracted from the image â†’
`extracted.get("patient_name")` | `"Ramesh Gupta"` | | `existing_patient` | MongoDB search â†’
`find_one({"name": ...})` | `{_id, name, visit
```

```
s, ...}` | | `visit` | Built from extracted data | `{date, doctor, medicines, ...}` | |
`patient_allergies` | Combined from existing + new â†’ `list(set(...))` | `["penicillin", "sulfa"]` |
| `new_medicines` | Extracted from image â†’ `extracted.get("medicines")` | `[{name, dose, ...}]` | |
`conflict_result` | Alert tool checks conflicts â†’ `check_drug_conflicts()` | `{has_alerts, alerts,
...}` | --- This should clarify every single line. Ask if you need more details on any specific part!
```